

Resources

1. <https://www.ibm.com/think/topics/natural-language-processing>
2. <https://medium.com/@maleeshadesilva21/preprocessing-steps-for-natural-language-processing-nlp-a-beginners-guide-d6d9bf7689c9>
3. <https://medium.com/@piyushkashyap045/understanding-tokenizers-in-natural-language-processing-nlp-1c295d3cfec2>
4. <https://medium.com/@tam.tamanna18/key-terms-and-concepts-in-natural-language-processing-14f7c1579de8>
5. <https://www.scaler.com/topics/nlp/text-representation-in-nlp/>
6. <https://medium.com/@harsh.vardhan7695/a-comprehensive-guide-to-word-embeddings-in-nlp-ee3f9e4663ed>

NLP (Natural Language Processing)

Natural language processing (NLP) is a subfield of computer science and artificial intelligence (AI) that uses machine learning to enable computers to understand and communicate with human language.

NLP enables computers and digital devices to recognize, understand and generate text and speech by combining computational linguistics, the rule-based modeling of human language together with statistical modeling, machine learning and deep learning.

Benefits of NLP

NLP makes it easier for humans to communicate and collaborate with machines, by allowing them to do so in the natural human language they use every day. This offers benefits across many industries and applications.

- Automation of repetitive tasks
- Improved data analysis and insights
- Enhanced search
- Content generation

Three different approaches to NLP include:

Rules-based NLP

The earliest NLP applications were simple if-then decision trees, requiring preprogrammed rules. They are only able to provide answers in response to specific prompts, such as the original version of Moviefone, which had rudimentary natural language generation (NLG)

capabilities. Because there is no machine learning or AI capability in rules-based NLP, this function is highly limited and not scalable.

Statistical NLP

Developed later, statistical NLP automatically extracts, classifies and labels elements of text and voice data and then assigns a statistical likelihood to each possible meaning of those elements. This relies on machine learning, enabling a sophisticated breakdown of linguistics such as part-of-speech tagging.

Statistical NLP introduced the essential technique of mapping language elements, such as words and grammatical rules to a vector representation so that language can be modeled by using mathematical (statistical) methods, including regression or Markov models. This informed early NLP developments such as spellcheckers and T9 texting (Text on 9 keys, to be used on Touch-Tone telephones).

Deep learning NLP

Recently, deep learning models have become the dominant mode of NLP, by using huge volumes of raw, unstructured data both text and voice to become ever more accurate. Deep learning can be viewed as a further evolution of statistical NLP, with the difference that it uses neural network models. There are several subcategories of models:

Sequence-to-Sequence (seq2seq) models: Based on recurrent neural networks (RNN), they have mostly been used for machine translation by converting a phrase from one domain (such as the German language) into the phrase of another domain (such as English).

Transformer models: They use tokenization of language (the position of each token words or subwords) and self-attention (capturing dependencies and relationships) to calculate the relation of different language parts to one another. Transformer models can be efficiently trained by using self-supervised learning on massive text databases. A landmark in transformer models was Google's bidirectional encoder representations from transformers (BERT), which became and remains the basis of how Google's search engine works.

Autoregressive models: This type of transformer model is trained specifically to predict the next word in a sequence, which represents a huge leap forward in the ability to generate text. Examples of autoregressive LLMs include GPT, Llama, Claude and the open-source Mistral.

Foundation models: Prebuilt and curated foundation models can speed the launching of an NLP effort and boost trust in its operation. For example, the IBM® Granite™ foundation models are widely applicable across industries. They support NLP tasks including content generation and insight extraction. Additionally, they facilitate retrieval-augmented generation, a framework for improving the quality of response by linking the model to external sources of knowledge. The models also perform named entity recognition which involves identifying and extracting key information in a text.

NLP Tasks

Several NLP tasks typically help process human text and voice data in ways that help the computer make sense of what it's ingesting. Some of these tasks include:

Coreference resolution
Named entity recognition
Part-of-speech tagging
Word sense disambiguation

How NLP works

1. Text preprocessing

Begins with tokenization, which involves splitting the text into smaller units like words, sentences or phrases. This helps break down complex text into manageable parts. Next, lowercasing is applied to standardize the text by converting all characters to lowercase.

Stop word removal is another common step, where frequently used words like "is" or "the" are filtered out because they don't add significant meaning to the text. Stemming or lemmatization reduces words to their root form (e.g., "running" becomes "run"), making it easier to analyze language by grouping different forms of the same word

2. Feature extraction

Feature extraction is the process of converting raw text into numerical representations that machines can analyze and interpret. This involves transforming text into structured data by using NLP techniques like Bag of Words and TF-IDF, which quantify the presence and importance of words in a document. More advanced methods include word embeddings like Word2Vec or GloVe, which represent words as dense vectors in a continuous space, capturing semantic relationships between words.

3. Text analysis

Text analysis involves interpreting and extracting meaningful information from text data through various computational techniques. This process includes tasks such as part-of-speech (POS) tagging, which identifies grammatical roles of words and named entity recognition (NER), which detects specific entities like names, locations and dates. Dependency parsing analyzes grammatical relationships between words to understand sentence structure, while sentiment analysis determines the emotional tone of the text, assessing whether it is positive, negative or neutral

4. Model training

Processed data is then used to train machine learning models, which learn patterns and relationships within the data. During training, the model adjusts its parameters to minimize errors and improve its performance. Once trained, the model can be used to make predictions or generate outputs on new, unseen

data. The effectiveness of NLP modeling is continually refined through evaluation, validation and fine-tuning to enhance accuracy and relevance in real-world applications.

Challenges of NLP

1. Biased training
2. Misinterpretation
3. New vocabulary
4. Tone of voice

Preprocessing Steps for Natural Language Processing (NLP)

Main text preprocessing techniques used in NLP are —

1. Text Cleaning

In this step, we will perform fundamental actions to clean the text. These actions involve transforming all the text to lowercase, eliminating characters that do not qualify as words or whitespace, as well as removing any numerical digits present.

a. Converting to lowercase

Python is a case sensitive programming language. Therefore, to avoid any issues and ensure consistency in the processing of the text, we convert all the text to lowercase.

```
df = df.applymap(lambda x: x.lower() if isinstance(x, str) else x)
```

b. Removing URLs

When building a model, URLs are typically not relevant and can be removed from the text data.

C. Removing non-word and non-whitespace characters

It is essential to remove any characters that are not considered as words or whitespace from the text dataset.

```
df = df.replace(to_replace=r'[^\\w\\s]', value='', regex=True)
```

D. Removing digits

It is important to remove all numerical digits from the text dataset. This is because, in most cases, numerical values do not provide any significant meaning to the text analysis process.

```
df = df.replace(to_replace=r'\d', value='', regex=True)
```

2. Tokenization

Tokenization is the process of breaking down large blocks of text such as paragraphs and sentences into smaller, more manageable units. By performing word tokenization, we can obtain a more accurate representation of the underlying patterns and trends present in the text data.

```
import nltk
from nltk.tokenize import word_tokenize

df['Message'] = df['Message'].apply(word_tokenize)
```

3. Stopword Removal

Stopwords refer to the most commonly occurring words in any natural language.

For the purpose of analyzing text data and building NLP models, these stopwords might not add much value to the meaning of the document. Therefore, removing stopwords can help us to focus on the most important information in the text and improve the accuracy of our analysis.

```
import nltk
from nltk.corpus import stopwords

stop_words = set(stopwords.words('english'))
df['Message'] = df['Message'].apply(lambda x: [word for word in x if word not in stop_words])
```

4. Stemming/Lemmatization

What's the difference between Stemming and Lemmatization?

Stemming is a simple and practical approach that involves cutting off the ends of words with the intention of obtaining the correct root form.	Lemmatization aims to remove inflectional endings only and to return the base or dictionary form of a word, which is known as the ' <i>lemma</i> '.
Eg: automate, automatic, automation → automat	Eg: car, cars, cars', car's → car am, is, are → be

There are various algorithms that can be used for stemming,

- Porter Stemmer algorithm
- Snowball Stemmer algorithm
- Lovins Stemmer algorithm

Stemming

Let's take a look at how we can use 'Porter Stemmer' algorithm on our dataset.

Some basic rules defined under the Porter Stemmer algorithm are,

sses → ss

Eg: caresses → caress

ies → i

Eg: ponies → poni

ational → ate

Eg: international → internate

tional → tion

Eg: conditional → condition

Types of Tokenization

1. Word-Based Tokenization

Word-based tokenizers split text into individual words. It's one of the simplest approaches to tokenization.

Example

Using Python's split() function:

```
text = "The quick brown fox jumps over the lazy dog"
tokenized_text = text.split()
print(tokenized_text)
```

Output:

```
['The', 'quick', 'brown', 'fox', 'jumps', 'over', 'the', 'lazy', 'dog']
```

Advantages

Easy to implement.

Retains meaningful units (words).

Disadvantages

Large vocabularies: The English language has over 500,000 words, leading to high storage requirements.

No semantic similarity: Words like “run” and “running” are treated as entirely different tokens.

Unknown tokens: Unrecognized words are replaced with special tokens like [UNK], leading to information loss.

2. Character-Based Tokenization

This approach breaks text into individual characters rather than words.

Example

Text: “The quick brown fox.”

Tokenized Output:

['T', 'h', 'e', ' ', 'q', 'u', 'i', 'c', 'k', ' ', 'b', 'r', 'o', 'w', 'n', ' ', 'f', 'o', 'x', '.']

Advantages

Smaller vocabularies: Works with only a limited set of characters.

Minimal unknown tokens: Rare words can still be represented since all words are built from characters.

Disadvantages

Higher computational load: A single word can result in multiple tokens.

Less semantic meaning: Individual characters lack standalone meaning, except in languages like Chinese where each character conveys significant information.

3. Subword Tokenization

Subword tokenizers find a middle ground between word- and character-based approaches. They split rare words into smaller, meaningful subwords while keeping frequent words intact.

Example

Text: "Tokenization simplifies NLP."

Tokenized Output:

['Token', '##ization', 'simplifies', 'N', '##LP', '.']

Here, "Tokenization" is split into "Token" and "##ization," while "NLP" is broken into "N" and "##LP."

Advantages

Efficient vocabularies: Balances between large word vocabularies and small character vocabularies.

Semantic preservation: Retains meaning through subword units.

Low unknown tokens: Covers most language constructs effectively.

Common Algorithms

Byte Pair Encoding (BPE): Used in GPT-2.

WordPiece: Used in BERT.

SentencePiece: Used in multilingual models.

Key Takeaways

Choose the right tokenizer: Use word-based tokenizers for simple tasks, character-based ones for low-resource languages, and subword tokenizers for balanced performance.

Understand the vocabulary: A tokenizer's vocabulary size impacts both memory usage and semantic coverage.

Use pretrained tools: Frameworks like Hugging Face provide ready-to-use tokenizers tailored to specific models.

Key Terms and Concepts in Natural Language Processing

Natural language processing (NLP) is a branch of artificial intelligence that deals with analyzing, understanding, and generating human language.

Here are some of the key terms and concepts in natural language processing (NLP):

Tokenization: Splitting text into individual words, phrases, symbols, or other meaningful elements called tokens. This is an essential first step for processing text.

Stop words: Common words like “a”, “and”, “the” that provide little informational value. Stop words are often filtered out before processing text.

Stemming: Reducing words to their root form. For example, “learning” is reduced to “learn”. This helps consolidate vocabulary and reduce sparsity.

Lemmatization: Similar to stemming but produces valid lemmas or dictionary roots. For example, “better” becomes “good” rather than just removing suffixes.

Bag-of-words: A representation of text that describes the occurrence of words within a document. The order of words is ignored.

N-grams: Sequences of n consecutive words used to understand word order and local context. For example, “artificial intelligence” becomes a 2-gram.

POS tagging: Assigning parts of speech (like noun, verb, adjective) to tokens in text. Useful for disambiguation.

Named entity recognition: Identifying and classifying key entities (like people, places, organizations) in text. Essential for understanding meaning.

Word embeddings: Representing each word as a high-dimensional numeric vector that encodes semantic meaning based on word usage. Enables mathematical operations.

Corpus: Collection of texts used for NLP analysis.

Syntax: Structure and grammar of sentences.

Semantics: Meaning of language.

Sentiment analysis: Identifying emotional tone behind text, like opinions, emotions, attitudes in text.

Named entity recognition: Finding and labeling entities like people, places.

Parsing: Analyzing sentence structure and relationship between words.

Co-reference resolution: Linking words or phrases that refer to the same entity.

Word sense disambiguation: Determining meaning of ambiguous words based on context.

Machine translation: Automated translation between languages.

Question answering: Systems that answer questions based on knowledge.

Dialog systems: Automated conversational agents (chatbots).

Spell correction: Spell correction identifies and automatically corrects misspellings in text using dictionaries or language models.

Optical character recognition (OCR): OCR converts images of text into machine-readable data, enabling text extraction from scanned documents, images, and videos.

Natural language generation: Natural language generation (NLG) creates textual content from data, aiming to produce coherent and contextually appropriate sentences.

Contextual embeddings: Contextual embeddings represent words in their surrounding context, enabling richer semantic understanding. Models like BERT and ELMo achieve this by learning from text.

Language models: Language models learn language structure and generate probable sequences of words. Examples include GPT-3, which can generate human-like text based on input prompts.

Probabilistic models: Probabilistic models use statistical methods and probabilities to model language, allowing for handling uncertainty in text data.

Linguistic rules: Linguistic rules encode grammar structures, morphology, and syntax into algorithms for language processing.

Ontologies: Ontologies are structured representations of concepts and relationships, often used to model knowledge and support semantic understanding.

Speech recognition: Speech recognition converts spoken language to text, enabling voice command and transcription applications.

Natural language understanding: Natural language understanding (NLU) derives meaning from text, interpreting user intents and making sense of linguistic input.

Discourse analysis: Discourse analysis studies language use in conversations and texts to understand how sentences and paragraphs connect to convey meaning.

Coreference: Determining when multiple expressions refer to the same entity.

Knowledge graphs: Knowledge graphs represent interconnected entities and concepts as a graph data model, often used in information retrieval and knowledge management.

Sequence-to-sequence models: Sequence-to-sequence models process both input and output sequences, used in tasks like translation, where the model transforms a source sequence into a target sequence.

Language vectors: Language vectors represent words or sentences as numerical vectors capturing semantics, allowing for mathematical operations like similarity calculations.

Transducers: Transducers are models that process input sequentially while producing output, useful for tasks like speech recognition and translation.

Morphology: Morphology studies the internal structure of words and how words are formed, including how roots, prefixes, and suffixes combine.

Phonology: Phonology examines the sound systems of languages, including phonemes and how they are organized and used.

Pragmatics: Pragmatics explores how context and intent influence language understanding, including how meaning changes based on context.

Discourse: Discourse involves connected language across multiple sentences or conversations, providing context for understanding longer texts.

Anaphora/Cataphora: Anaphora and cataphora involve linking pronouns or nouns back (anaphora) or forward (cataphora) to other words or phrases for clarity.

Topic modeling: Discovering abstract topics in a collection of documents.

Information extraction: Extracting structured info like entities, relationships, events from text.

Summarization: Generating a summary capturing the main points of a larger text.

Question generation: Automatically creating questions from passages of text.

Coreference resolution: Determining when two expressions refer to the same entity.

Word sense disambiguation: Identifying the correct sense of ambiguous words.

Optical character recognition: Converting images of text into machine-readable text.

Multimodal learning: Combining language with other modes like vision or speech.

Phonetics: The study of speech sounds and pronunciation.

Phonology: The study of the sound systems of languages.

Morphophonology: Studying how morphology interacts with phonology.

Sociolinguistics: Studying language in social contexts.

Forensic linguistics: Forensic linguistics applies linguistic analysis in legal contexts, aiding in areas such as authorship attribution and linguistic profiling.

Psycholinguistics: Psycholinguistics studies the psychological processes of language use, including how language is acquired, comprehended, and produced.

Neurolinguistics: Neurolinguistics examines the neuroscience of language processing in the brain, exploring how language and cognition are connected.

Stylistics: Stylistics analyzes and understands linguistic style, exploring how language is used to create tone, mood, and impact.

Readability: Measuring how easy a text is to read and understand.

Text simplification: Reducing complexity of text to make it more readable.

Code-switching: Code-switching involves switching between languages or dialects in speech or writing, often reflecting bilingual or multilingual speakers' linguistic flexibility.

Multiword expressions: Words that act as a single semantic unit like “case study”.

Evaluation metrics: Ways to quantitatively evaluate NLP systems like BLEU, ROUGE.

Low resource languages: Low-resource languages have limited digital data available, making NLP tasks more challenging due to the lack of training resources.

Common Terms Used While Representing Text in NLP

Corpus(C): All the text data or records of the dataset together are known as a corpus.

Vocabulary(V): This consists of all the unique words present in the corpus.

Document(D): One single text record of the dataset is a Document.

Word(W): The words present in the vocabulary.

Types of Text Representation in NLP

Text representation can be classified into two types:-

- Discrete text representation
- Distributed text representation

In discrete text representation, the individual words in the corpus are mutually exclusive and independent of one another. Eg: One Hot Encoding. Distributed text representation thrives on the co-dependence of words in the corpus. The information about a word is spread along the vector which represents it. Eg: Word Embeddings.

This article focuses on the discrete text representation in NLP. Some of them are:-

- One Hot Encoding
- Bag of words
- Count Vectorizer
- TF-IDF
- Ngrams

1. One Hot Encoding

In this representation of NLP, every word in a text corpus is assigned a vector that consists of 0 and 1. This vector is termed the one hot vector in NLP. Every word is assigned a unique hot vector. This allows the machine learning model to recognize each word uniquely through its vector. The one hot vector contains a single "1", the rest all being zeroes. The "1" here is present in the position corresponding to the position of the word in the sentence or in the vocabulary.

Advantages

- Easy to understand and implement

Disadvantages

- We get a highly sparse matrix with each sentence, the majority of the values are zero.
- If the documents are of different sizes, we get different-sized vectors.
- This representation does not capture the semantic meaning of the words.

2. Bag of Words

This representation, unlike one hot encoding, converts a whole piece of text into fixed-length vectors. This is done by counting the number of times a word has appeared in the document. The frequency count of words helps us to compare and contrast documents. This representation has a variety of applications such as topic modeling, document classification, spam email detection, and many more.

On an elementary level, the bag of words is implemented using CountVectorizer.

CountVectorizer

CountVectorizer implements the bag words concept by computing the frequency of each word in the corpus and creating a matrix with the

sentences(or their position) as rows, and words as columns and filling it with the words' corresponding frequencies.

Advantages of CountVectorizer

1. Easy to implement
2. We get the frequency of each word
3. The size of the vector is the size of the dictionary.

Disadvantages of CountVectorizer

1. We do not extract any semantic meaning from the words.
2. This method also loses the positional information of a word.
3. High-frequency words do not necessarily have the highest weightage.

On an advanced level, the bag of words is implemented using TF-IDF representation.

TF-IDF

This representation of text in NLP overcomes some of the drawbacks of **CountVectorizer**. There is a need to suppress high-frequency words and ignore very low-frequency words so that we can normalize the weights. The word which appears multiple times in a document and rarely in the corpus should have a good weightage. This weightage is calculated using two formulas:- Term Frequency and Inverse Document Frequency.

Term Frequency: The ratio of the number of occurrences of a word in a document to the number of terms in the document is called term frequency.

$tf(t,d)=$

$tf(t,d)=$ count of

t

t in

d

d / number of words in

d

d

" t " is the term and " d " is the number of terms in the .document The tf of the word "love" in the sentence "I love nlp" is 1/3.

Inverse document frequency: Idf of a term is the **log of the ratio** of the number of documents in the corpus to the number of documents with the term in them.

$$\text{idf}(t) = \log(N/(df+1))$$

$$\text{idf}(t) = \log(N/(df+1))$$

N is the number of documents and df is the number of documents with the term t . We add 1 to the denominator of the fraction to avoid division by 0. We multiply tf and idf to get the **tf-idf** weightage.

The **tf-idf** of the term t in document d is:

$$\text{tf-idf}(t,d) = \text{tf}(t,d) * \log(N/(df+1))$$

$$\text{tf-idf}(t,d) = \text{tf}(t,d) * \log(N/(df+1))$$

Advantages

- Easy to understand and interpret.

- Penalizes both high and low-frequency words.

Disadvantages

- We do not capture the positional information of a word.
- Semantic meanings of the words are not captured.

Ngrams

This representation is similar to the `CountVecorizer` bag words model, the only difference is that instead of calculating the frequency of one word we calculate the frequency of groups of words(in groups of two or more) occurring together in the corpus. In this way, we get a better understanding of the text. Based on the number of words grouped together, the model has termed a bigram (2 words), trigram(3 words), and so on.

Notice that for the first statement the phrase "nlp has" with key 5, has contributed 1 to the fifth index in the resultant vector.

Advantages of Ngrams

- Ngrams capture the semantic meaning of the sentence and help in finding the relationship between words.
- Easy to implement

Disadvantages

- Out of vocabulary(OOV): We cannot capture the relationships between the words or their semantic meanings if they are not in the vocabulary

Linear Text Classification

Text classification is the supervised learning method of categorizing pieces of text available in the dataset. We assign labels to the text on the basis of their genre, specialty, etc. We can use text classification for sentiment analysis, topic detection, spam email detection, language detection, and many more.

We are going to see how classification models like RandomForest are implemented and how TF-IDF vectorizer is used.

We use the "Disaster tweet dataset from Kaggle", where each tweet is labeled as 1 if it is about a disaster and 0 otherwise.

Conclusion

This article was all about text representation in NLP. Here are the key takeaways from this article:-

- Representing text in a particular manner is very crucial before training a machine learning model.
- The more complex the representation gets, the more accuracy and results of the model get better.
- We learned about discrete text representations such as one hot encoding, bag of words, tf-idf, etc. and how they are implemented.
- We saw how one hot encoding encodes each word into a sparse vector and how countVectorizer uses the frequency of each word to encode it, squeezing the entire sentence into a single vector.
- Tf-Idf improves the countVectorizer, by attempting to normalize the weightage of all the words in the corpus.
- Finally we saw the different methods to classify text.